

Desenvolvimento de Drivers e Integração com Hardware

Objetivo

- Simular a leitura de dados de sensores do veículo (ex: velocidade, temperatura) e a interação com o sistema embarcado através de um “driver” em user- space.
- Criar uma simulação conceitual da comunicação CAN, demonstrando o envio e recebimento de mensagens e sua aplicação no controle do equalizador.

Link do Repositório: [gitHub](#)

Link da Aplicação: [CANSimulationBUS](#)

Entregáveis

- Código-fonte do driver simulado (VehicleSensorSimulator.kt).
- Código-fonte da simulação CAN (CanMessage.kt, VehicleCanBusSimulator.kt).
- Guia explicativo sobre a integração do driver e da comunicação CAN (este e-book).

Estrutura de Pastas do Aplicativo

```
.
├── com
│   └── leandromendes
│       └── cansimulationbus
│           ├── data
│           │   └── model
│           │       └── CanMessage.kt
│           ├── MainActivity.kt
│           └── util
│               ├── VehicleCanBusSimulator.kt
│               └── VehicleSensorSimulator.kt
```

CanMessage.kt: Classe de dados que representar uma mensagem CAN.

VehicleCanBusSimulator.kt: Esta classe simulará o barramento CAN do veículo, permitindo o envio e recebimento de mensagens. Ela também terá um mecanismo para notificar a MainActivity sobre mensagens recebidas, como um novo nível de volume.

Detalhes:

- Usando `kotlinx.coroutines` e `Channel` para simular um fluxo assíncrono de mensagens, como ocorreria em um barramento CAN real.
- `sendMessage` simula o envio de uma mensagem para o barramento.
- O bloco `init` lança uma corrotina que “escuta” o canal de mensagens, simulando um receptor CAN. Quando uma mensagem é recebida, ela é processada (ex: extraíndo o volume) e também emitida

através de um `MutableSharedFlow(_canMessageFlow)`.

- O `MutableSharedFlow` permite que múltiplos observadores (como a `MainActivity`) recebam as mensagens CAN sem que o simulador precise saber quem são esses observadores. Isso é um padrão de design robusto para comunicação assíncrona.

VehicleSensorSimulator.kt: A classe `VehicleSensorSimulator` simula um sensor veicular genérico. O método `readSensorData()` retorna um valor aleatório baseado no tipo de sensor, e `calibrateSensor()` simula uma operação de calibração.

Resultados

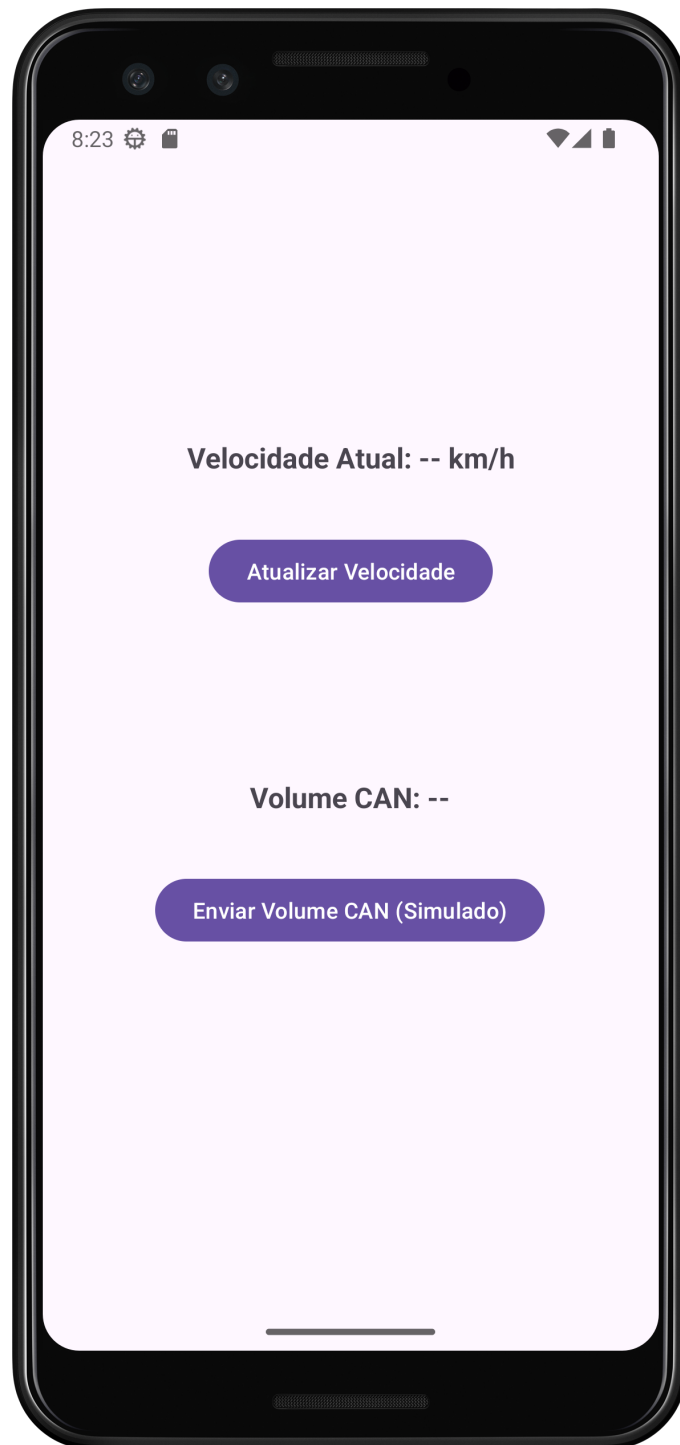


Figura 1: Tela inicial do aplicativo



Figura 2: Tela do aplicativo após apertar os botões

```
20:23:51.308 7287-7287 CANSimulationBus com.leandromendes.cansimulationbus D [Speed] Lendo dados do sensor: 139
20:23:51.309 7287-7287 CANSimulationBus com.leandromendes.cansimulationbus D Velocidade lida: 139 km/h
```

Figura 3: Log de execução da mensagem de velocidade

```
20:24:33.712 7287-7312 CANSimulationBus com.leandromendes.cansimulationbus I Enviando mensagem CAN (ID: 0x123, Dados: 61)
20:24:33.713 7287-7311 CANSimulationBus com.leandromendes.cansimulationbus D Mensagem CAN recebida (ID: 0x123, Dados: 61)
20:24:33.716 7287-7311 CANSimulationBus com.leandromendes.cansimulationbus I Processando mensagem CAN: Novo Volume Mestre: 97
```

Figura 4: Log de execução da mensagem de alteração de volume.